

SYSTEM / OPERATIONS / RUNBOOK

# HH AGENT

Автономный агент поиска, оценки и контролируемого отклика на вакансии.

---

**Repository**

Xander-Rude/hh-agent

---

**Baseline**

main @ 241fc51

---

**Updated**

25.08.2026



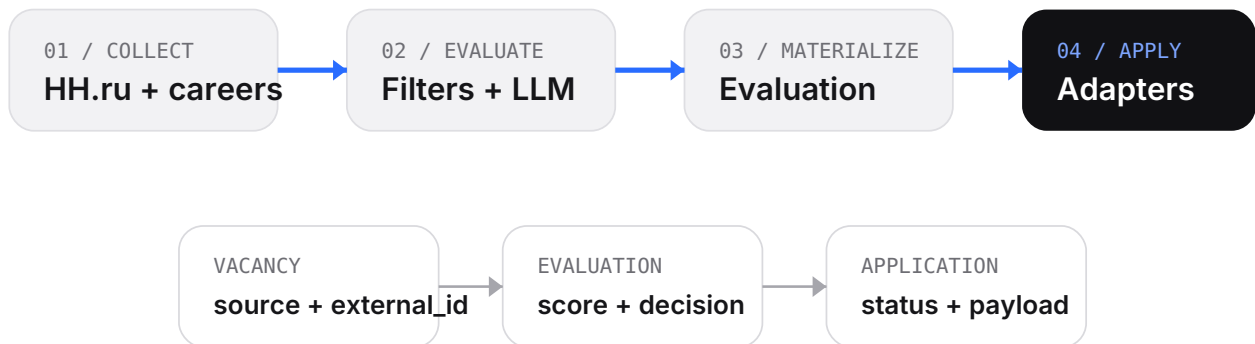
# Как устроен агент

Документ описывает рабочую систему как она запускается на Windows-хосте: от сбора вакансии до оценки, выбора резюме, очереди отклика и диагностики инцидентов.

|    |                           |    |    |                               |    |
|----|---------------------------|----|----|-------------------------------|----|
| 01 | Архитектура и источники   | 03 | 02 | Оценка вакансий               | 05 |
| 03 | Резюме и сопроводительное | 06 | 04 | Модель данных и статусы       | 07 |
| 05 | HH apply                  | 08 | 06 | Yandex apply                  | 09 |
| 07 | Windows Scheduler         | 10 | 08 | Runtime, lock, profiles, logs | 11 |
| 09 | Telegram                  | 12 | 10 | Environment                   | 13 |
| 11 | Операционный runbook      | 14 | 12 | Incident playbooks            | 15 |
| 13 | Safety invariants         | 16 | 14 | Карта репозитория             | 17 |
| 15 | Расширение и техдолг      | 18 | A  | Cheat sheet                   | 19 |

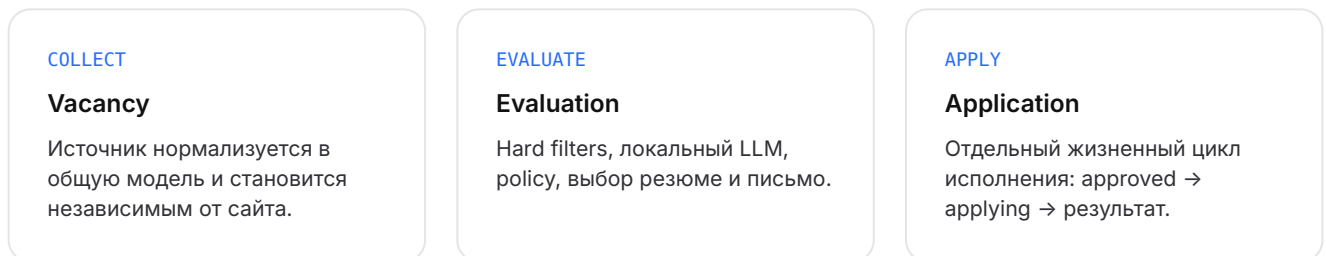
# Конвейер, а не набор кликов

Сбор, оценка и отправка разделены. UI конкретного сайта появляется только на границах collector/apply adapter; между ними работает общий контракт данных.



## COMMON CONTRACT

База отделяет логику принятия решения от UI конкретного сайта.



# Два источника, разные способы сбора

HH.RU

## Browser collector

Playwright + persistent profile.

HH collector работает через браузер и использует общий browser-profile. Вакансии сохраняются как source=hh; legacy hh\_id поддерживается для обратной совместимости.

```
hh_collect.py
-> browser-profile
-> Vacancy(source="hh", external_id=hh_id)
```

YANDEX JOBS

## HTTP collector

Без Playwright на этапе сбора.

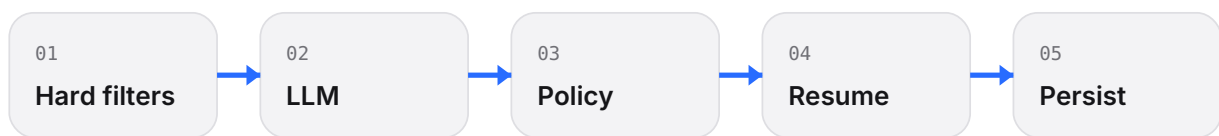
sources/yandex.py использует httpx и HTMLParser, собирая целевые категории project/technical management. UI браузера нужен только позже, при реальном отклике.

```
collect_careers.py
-> sources/yandex.py
-> RawVacancy
-> Vacancy(source="yandex", external_id=...)
```

Новый корпоративный источник должен нормализоваться в Vacancy и не тянуть site-specific логику внутрь evaluator.

# От фильтра к решению

Оценка строится слоями: однозначные запреты применяются до LLM, затем модель даёт structured evaluation, а policy исправляет системные противоречия и пересчитывает итог.



SCORE    role .35 + seniority .20 + domain .15 + responsibility .30

## Hard filters

Зарплата, нежелательные домены, стоп-слова, role coverage и явный запрет internship/junior.

## LLM

Локальная Ollama. По умолчанию gemma3:12b-it-gat, temperature 0.2.

## Management policy

Детерминированные floors и очистка ложных negative markers для управленческих ролей.

## Decision thresholds

**apply** — SCORE\_THRESHOLD, default 80.

**review** — HH\_REVIEW\_THRESHOLD, default 70 и не выше apply threshold.

**reject** — ниже порога либо есть существенные red flags.

# Контент готовится до адаптера

HH и Yandex получают уже выбранное резюме и готовое сопроводительное. Адаптер не решает, что писать — он только доставляет payload в интерфейс сайта.

## RESUME MATCHER

### Выбор существующего резюме

Использует vacancy title/description и итоговый score. Возвращает key, title, resume id и match score.

## COVER LETTER

### Vacancy-aware, без выдуманного опыта

Берёт подтверждённые факты профиля и связывает их с конкретным фокусом вакансии.

## Для Yandex

Перед каждым откликом worker повторно загружает выбранный локальный PDF. Файл в профиле не считается источником истины.

```
data/resumes/*.pdf
↓
selected_resume_key
↓
Yandex profile upload
```

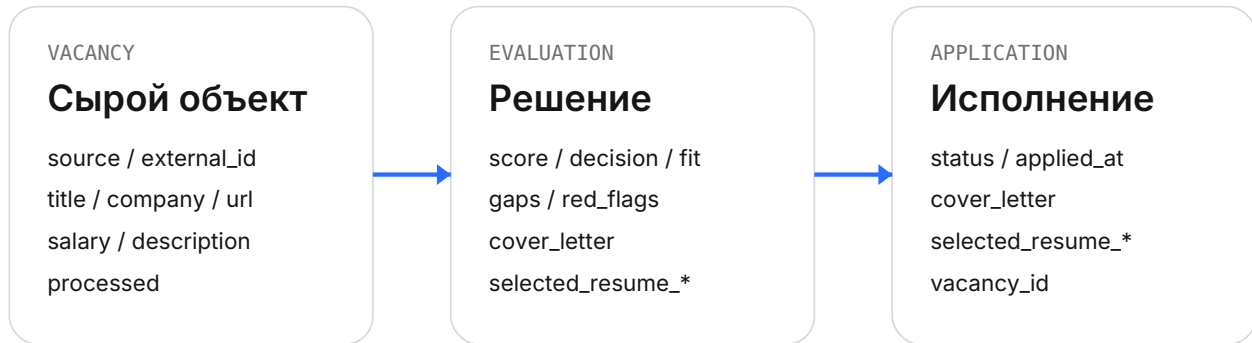
## Для HH

Application хранит выбранное резюме и письмо; отдельный HH worker отправляет approved-очередь по своему расписанию.

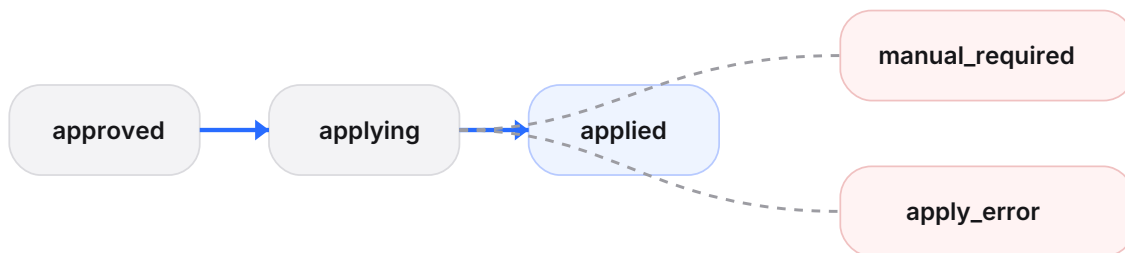
Презентация в Yandex-flow не используется: в форме нет отдельного поля загрузки.

## 04 / DATA MODEL

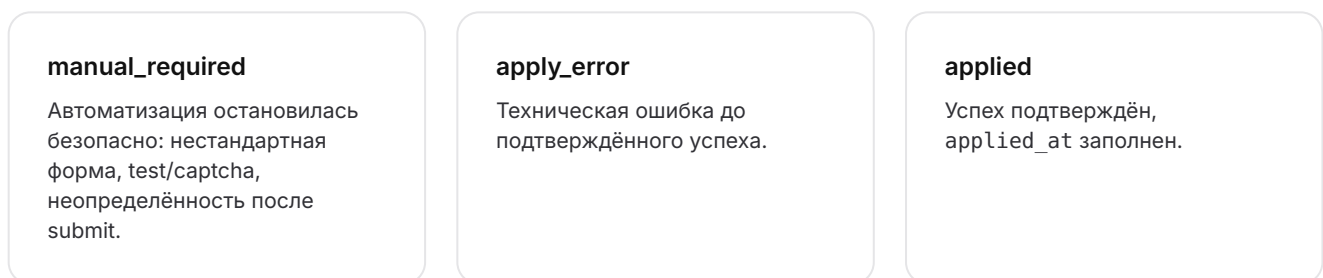
# Vacancy → Evaluation → Application



**HISTORY** Evaluation не перезаписывается; safety-проверки используют latest и сохраняют историю.



**NO BLIND RETRY** После submit без подтверждения успеха — manual\_required, а не повторная отправка.



# Отдельный worker, отдельное расписание

## ENTRY POINT

### background\_apply.py

Supervisor получает общий AgentLock, запускает `apply_worker.py` с `HH_APPLY_HEADLESS=true`, пишет runtime state и логирует дочерний процесс.

```
HH Agent - Apply
-> pythonw.exe background_apply.py
-> AgentLock
-> apply_worker.py
-> approved HH Applications
```

## QUEUE

Application.status=approved и источник HH.

## CADENCE

Windows Scheduler — каждые 10 минут.

Текущий scheduled Yandex stage отключает HH dispatch через `APPLY_DISPATCH_HH=false`, поэтому два apply-flow не смешиваются.

# Автоматически — только decision=apply

Yandex встроен в 4-й этап основного pipeline. Dispatcher дополнительно проверяет latest Evaluation и не пропускает review/reject даже при старом status=approved.

```
background_pipeline.py
-> apply_dispatcher.py
    APPLY_DISPATCH_HH=false
    YANDEX_APPLY_LIVE=true
    YANDEX_APPLY_HEADLESS=true
-> yandex_apply_worker.py
```

01

## Resume

Удалить текущий файл из формы и загрузить локальный выбранный PDF.

02

## Form

Заполнить cover letter и только обязательное consent.

03

## Submit

Нажать только если стандартная форма готова и нет manual markers.

Если submit уже был нажат, но success не подтверждён — manual\_required. Автоматического повторного submit нет.

# Четыре фоновых контура



**AgentLock** Если job пересекаются, второй запуск завершается как skipped вместо второго browser process.

| TASK                    | ACTION                                 | SCHEDULE                             |
|-------------------------|----------------------------------------|--------------------------------------|
| HH Agent - Pipeline     | pythonw.exe background_pipeline.py     | каждые 30 мин; старт 00:03           |
| HH Agent - Apply        | pythonw.exe background_apply.py        | каждые 10 мин; старт 00:08           |
| HH Agent - Resume Raise | pythonw.exe background_resume_raise.py | каждые 30 мин; 00:23                 |
| HH Agent - Telegram     | Telegram launcher                      | при входе; restart 1 мин при падении |

Pipeline, HH Apply и Resume Raise делят один AgentLock. Пересечение расписаний допустимо: второй job завершается как skipped.

# Lock, профили и журналы

## Persistent browser profiles

HH — C:\hh-agent\browser-profile

Yandex — C:\hh-agent\yandex-browser-profile

## Runtime state

data/runtime/pipeline.json

data/runtime/apply.json

data/runtime/resume\_raise.json

data/runtime/telegram.json

## Основные логи

logs/pipeline\_supervisor.log

logs/collector.log

logs/careers\_collector.log

logs/processor.log

logs/apply\_dispatcher.log

logs/yandex\_apply\_worker\*.log

logs/apply\_worker\_runtime.log

logs/resume\_raise\_worker.log

```
Get-Content C:\hh-agent\logs\pipeline_supervisor.log -Tail 100
Get-Content C:\hh-agent\logs\pply_dispatcher.log -Tail 150
Get-Content C:\hh-agent\data
untime\pipeline.json
```

Supervisor ставит hard timeout. При зависании дочерний process tree завершается, чтобы lock гарантированно освободился.

# Control plane

Telegram — не отдельный источник данных, а операционный интерфейс поверх runtime state и SQLite.

## `/health`

Файлы, Ollama, runtime states и counters.

## `/status`

PIPELINE / APPLY / RESUME  
RAISE и очередь.

## `/run`

Запустить  
background\_pipeline.py без  
консоли.

## `/new`

Показать новые вакансии.

## `/stats`

Application statuses.

## `INLINE`

Approve / skip / company  
blacklist.

## Single instance

Bot держит Windows file lock и завершает второй экземпляр, предотвращая Telegram Conflict по getUpdates.

## Security boundary

Текущая реализация принимает команды из любого Telegram chat. TELEGRAM\_CHAT\_ID используется как fallback для background notifications, а не как allowlist.

# Переменные, меняющие поведение

| VARIABLE                      | MEANING                             |
|-------------------------------|-------------------------------------|
| LLM_PROVIDER                  | Сейчас поддержан ollama.            |
| LLM_MODEL                     | Default gemma3:12b-it-qat.          |
| LLM_BASE_URL                  | Default http://localhost:11434.     |
| LLM_TIMEOUT / LLM_MAX_RETRIES | 120 сек / 2 retry по умолчанию.     |
| SCORE_THRESHOLD               | Apply threshold, default 80.        |
| HH_REVIEW_THRESHOLD           | Review threshold, default 70.       |
| HH_COLLECT_HEADLESS           | Pipeline выставляет true.           |
| HH_APPLY_HEADLESS             | background_apply выставляет true.   |
| YANDEX_APPLY_HEADLESS         | Pipeline выставляет true.           |
| YANDEX_APPLY_MAX_PER_RUN      | Default 5.                          |
| YANDEX_APPLY_DELAY_SECONDS    | Default 3.                          |
| YANDEX_APPLY_LIVE             | Боевой gate Yandex submit.          |
| YANDEX_APPLY_APPLICATION_ID   | Точечный target для controlled run. |
| APPLY_DISPATCH_HH             | На Yandex stage = false.            |
| TELEGRAM_BOT_TOKEN            | Секрет для bot.                     |
| TELEGRAM_CHAT_ID              | Fallback destination.               |
| TELEGRAM_MIN_SCORE            | Default 72.                         |

В документацию и README входят только имена переменных и семантика. Значения токенов и credentials не публикуются.

# Ежедневные команды

## Полный pipeline — PowerShell

```
cd C:\hh-agent
.\.venv\Scripts\python.exe .\background_pipeline.py
```

## Git Bash

```
cd /c/hh-agent
./venv/Scripts/python.exe ./background_pipeline.py
```

## Сессии

```
.\.venv\Scripts\python.exe .\check_hh_session.py
.\.venv\Scripts\python.exe .\check_yandex_session.py
```

## Scheduler

```
Get-ScheduledTask | Where-Object {$_.TaskName -like "*HH*"} | Select TaskName,State
Get-ScheduledTaskInfo -TaskName "HH Agent - Pipeline"
Start-ScheduledTask -TaskName "HH Agent - Pipeline"
```

## Обновить main

```
git checkout main
git pull origin main
```

В PowerShell используется C:\hh-agent. Путь /c/hh-agent относится только к Git Bash.

# Сначала read-only диагностика

## YANDEX / manual\_required

### 1. Проверить DB state

```
.\.venv\Scripts\python.exe .  
\inspect_application.py 212
```

### 2. Проверить worker history

```
.\.venv\Scripts\python.exe .  
\inspect_yandex_apply_log.py 212
```

### 3. Проверить сайт без submit

```
.\.venv\Scripts\python.exe .  
\probe_yandex_application_state.py 212
```

## AUTH / PIPELINE

### Session expired

```
.\.venv\Scripts\python.exe .\hh_login.py  
.\.venv\Scripts\python.exe .  
\yandex_login.py
```

### Pipeline skipped

last\_error=agent\_lock\_busy означает штатную взаимную блокировку.

#### Timeout

Supervisor завершает process tree и пишет exit code 124.

Requeue делается только после подтверждения, что на сайте отклик действительно не отправлен и обычная кнопка apply всё ещё доступна.

# Шесть правил, которые нельзя ломать

## 01 / SOURCE

### HH ≠ Yandex

Worker не должен получать Applications чужого source.

## 02 / APPLY ONLY

### Yandex auto = decision=apply

approved недостаточно; latest Evaluation обязан быть apply.

## 03 / NO RETRY

### Не повторять неопределённый submit

После нажатого submit без success — manual\_required.

## 04 / LOCAL RESUME

### PDF на диске — source of truth

Yandex re-upload перед каждым откликом.

## 05 / DB HISTORY

### Не стирать Evaluation

История нужна для аудита, latest decision и расследований.

## 06 / AGENTLOCK

### Browser jobs не параллелить

Один общий lock важнее частоты Scheduler.

# Что считать production

**app/db.py** — SQLAlchemy + SQLite + лёгкие миграции.

**app/evaluator.py** — structured LLM evaluation.

**app/evaluation\_policy.py** — management policy и cover fallback.

**app/hard\_filters.py** — быстрые запреты.

**app/resume\_matcher.py** — выбор резюме.

**process\_vacancies.py** — orchestration evaluation.

**apply\_worker.py** — HH apply.

**apply\_dispatcher.py** — source-aware routing.

**sources/base.py** — контракт source.

**sources/yandex.py** — Yandex HTTP collector.

**yandex\_apply\_worker.py** — Yandex production submit.

**background\_common.py** — lock/state/logging/timeouts.

**background\_pipeline.py** — 4-stage scheduled pipeline.

**background\_apply.py** — HH supervisor.

**background\_resume\_raise.py** — resume raise supervisor.

**telegram\_bot.py** — operator UI.

`inspect_*`, `probe_*`, session checks и guarded requeue — эксплуатационные инструменты. Они не входят в основной pipeline, но нужны для инцидентов.

# Как расширять систему

## Новый карьерный source

```
class NewCompanySource(VacancySource):
    name = "new_company"
    def collect(self) -> SourceResult:
        return SourceResult(
            vacancies=[RawVacancy(...)]
        )
```

Evaluation и БД не должны знать HTML/API нового сайта. Для auto-apply создаётся отдельный worker со своими guards.

## Технический долг

**Yandex dry\_run modules** — production worker импортирует helpers из файлов с историческими именами.

**resume\_raise\_worker.py + v2** — supervisor использует v2; старую версию нужно проверить на зависимости.

**Telegram public mode** — нет chat allowlist.

**Scheduler installers** — возможен drift между install scripts и фактическими actions на хосте.

**Tests/CI** — нет полного набора regression tests для policy, routing и browser helpers.

## Перед merge изменений apply/evaluator





# Быстрый доступ

## Диагностика Application

```
cd C:\hh-agent
.\.venv\Scripts\python.exe .
\inspect_application.py 212
.\.venv\Scripts\python.exe .
\probe_yandex_application_state.py 212
```

## Cover letter из SQLite

```
./venv/Scripts/python.exe -c "from app.db
import SessionLocal,Application;
s=SessionLocal(); a=s.get(Application,212);
print(a.cover_letter if a else 'not found');
s.close()"
```

## Targeted Yandex live — Git Bash

```
cd /c/hh-agent
YANDEX_APPLY_LIVE=true \
YANDEX_APPLY_APPLICATION_ID=212 \
./venv/Scripts/python.exe ./apply_dispatcher.py
```

## Проверить Scheduler actions

```
(Get-ScheduledTask -TaskName "HH Agent -
Pipeline").Actions
(Get-ScheduledTask -TaskName "HH Agent -
Apply").Actions
```

---

### 30m

Pipeline cadence

---

### 10m

HH Apply cadence

---

### 5

Yandex default max/run

Source baseline: Xander-Rude/hh-agent, main @ 241fc51. При изменении scheduler, status model, safety gates или source adapters документация должна обновляться в той же release-итерации.